Especificação SDD TaskAnalyzer

Especificação Técnica e Governança de Contexto (SDD e AI Harness).

Autor	Lucas Amaral Evangelista, RA 22508120
Curso	Ciência da Computação, CEUB, turma UN 0726
Disciplina	Bootcamp III
Versão	**1.2**, 18 de outubro de 2026
Versões anteriores	1.0, 21/08/2026, Fase 1; 1.1, 29/09/2026, Fase 2

Este documento é o **contrato executável** do módulo TaskAnalyzer. Ele é a fonte da verdade para o código gerado por assistentes de inteligência artificial. **Tudo o que não estiver escrito aqui não deve ser implementado.**

0. Histórico de versões

Versão	Data	Resumo
1.0	21/08/2026	Contrato original da Fase 1
1.1	29/09/2026	Alinhamento ao enunciado da Fase 2, detalhado na seção 0.1
1.2	18/10/2026	Infraestrutura da Fase 3: CI, Docker e artefatos de especificação, detalhado na seção 0.2

0.1 Mudanças da versão 1.1

O enunciado da Fase 2 fixou nomes, unidades e comportamentos diferentes dos definidos na versão 1.0. Como a especificação é a fonte da verdade, **o contrato foi revisado antes de qualquer geração de código**, e não o contrário.

ID	Na versão 1.0	Na versão 1.1	Motivo
MUD-01	função `analisar_tarefas`	função `analyze_tasks`	nome exigido pelo enunciado
MUD-02	exceções `TarefaInvalidaError` e `SemTarefasConcluidasError`, sob `TaskAnalyzerError`	exceção única `TaskValidationError`	nome exigido pelo enunciado
MUD-03	lista vazia ou sem tarefa concluída levanta exceção (RN-09)	lista vazia ou sem tarefa concluída devolve `0.0` nas métricas, sem exceção	comportamento exigido para os casos de borda
MUD-04	tempo médio em minutos	tempo médio em **horas**	campo `tempo_medio_conclusao_horas` exigido pelo enunciado

ID	Na versão 1.0	Na versão 1.1	Motivo
MUD-05	saída com `quantidade_tarefas` e `indicadores_por_prioridade`	saída com `total_tarefas`, `total_concluidas`, `total_pendentes`, `tempo_medio_conclusao_horas`, `tempo_medio_por_prioridade_horas` e `taxa_atraso_percentual`	campos exigidos pelo enunciado
MUD-06	prioridade sem tarefa concluída fica fora do resultado (RN-07)	as três prioridades aparecem sempre, com `0.0` quando não há concluída	coerência com MUD-03
MUD-07	saída como dataclass `MetricasProdutividade`	saída como dicionário tipado, `ResultadoAnalise` (`TypedDict`)	permite conferir campo a campo nos testes, mantendo tipagem
MUD-08	cenários CA-01 a CA-08	Cenário 1, Cenário 2, casos de borda e cenários complementares	numeração do enunciado
MUD-09	branches `main`, `develop` e `feature/<issue>-<slug>`	branches `main` e `feature/<nome>`	estratégia pedida no enunciado
MUD-10	nomes sempre em português	nomes públicos exigidos pelo enunciado ficam em inglês; o restante segue em português	conflito entre diretriz e enunciado
MUD-11	árvore sem arquivo de configuração	`pyproject.toml` na raiz	o pytest precisa saber que a raiz contém o pacote `src`

Sobre a MUD-03. A versão 1.0 levantava exceção para conjunto sem tarefa concluída porque, só com a média, um zero seria indistinguível de um desempenho real igual a zero. Na versão 1.1 essa ambiguidade é resolvida por outro caminho: o resultado traz `total_concluidas`, então `total_concluidas == 0` diz explicitamente que não havia base de cálculo. A proteção contra divisão por zero continua obrigatória (RN-08): muda apenas o que a função faz quando o denominador é zero.

Autorização. Mudanças autorizadas por Lucas Amaral Evangelista, dono do contrato, em 29/09/2026, antes da geração do código, conforme a proibição 5 da seção 3.2 da versão 1.0: não alterar assinatura, nome ou parâmetros públicos sem autorização registrada.

0.2 Mudanças da versão 1.2

A Fase 3 não muda nenhuma regra de negócio, nenhum campo e nenhuma assinatura. Ela muda **como o contrato é verificado e entregue**: o Test Harness passa a rodar sozinho a cada mudança e dentro de um ambiente empacotado.

ID	Na versão 1.1	Na versão 1.2	Motivo
MUD-12	branches `main` e `feature/<nome>`	branches `main`, `develop` e `feature/<nome>`; as features entram na `develop` e a `develop` entra na `main`	gatilhos da CI exigidos pelo enunciado da Fase 3, em `main` e `develop`
MUD-13	o harness roda só na máquina do desenvolvedor	o harness roda no GitHub Actions a cada push e Pull Request, em `ubuntu-latest` com Python 3.11	integração contínua (seção 4.4)

ID	Na versão 1.1	Na versão 1.2	Motivo
MUD-14	sem empacotamento	`Dockerfile` com `python:3.11-slim` e `docker-compose.yml` com o serviço `tests`	execução reprodutível em qualquer máquina (seção 4.5)
MUD-15	especificação apenas em Markdown	a especificação ganha duas formas derivadas: `specs/system_design_document.pdf`, para leitura, e `specs/requirements.yaml`, para leitura por máquina	arquitetura de repositório exigida pelo enunciado da Fase 3

O arquivo `task_analyzer_spec.md` continua sendo a fonte da verdade. O PDF e o YAML são gerados a partir dele e não podem divergir: se divergirem, vale o Markdown.

Autorização. Mudanças autorizadas por Lucas Amaral Evangelista em 18/10/2026.

---

1. Visão geral e contrato de negócio

1.1 Propósito do módulo

O TaskAnalyzer é um módulo de análise de tarefas e produtividade. Ele recebe um conjunto de tarefas já registradas e devolve um retrato quantitativo de como esse conjunto foi executado, sem opinar sobre pessoas e sem inferir nada que os dados não sustentem.

O módulo responde:

- quantas tarefas existem, quantas foram concluídas e quantas estão pendentes;
- qual o tempo médio de conclusão, em horas, no geral e por prioridade;
- qual o percentual de tarefas concluídas depois do prazo.

O escopo é deliberadamente estreito. O TaskAnalyzer calcula e devolve métricas. Ele não armazena dados, não desenha gráficos, não envia notificação e não decide nada por ninguém.

1.2 Contrato executável de interface

A assinatura pública abaixo não pode ser alterada pelo assistente de inteligência artificial sem autorização registrada:

```
def analyze_tasks(tarefas: list[Tarefa]) → ResultadoAnalise:
    """Calcula métricas de produtividade a partir de um conjunto de tarefas."""
```

O módulo `src/task_analyzer.py` define, além da função:

Nome	Tipo	Papel
`Tarefa`	dataclass congelada (`frozen=True`)	uma tarefa de entrada, com os campos da seção 1.2.1
`ResultadoAnalise`	`TypedDict`	o resultado, com os campos da seção 1.2.2
`TaskValidationError`	exceção, subclasse de `ValueError`	qualquer entrada que viole o contrato

Nome	Tipo	Papel
`analyze_tasks`	função	o ponto de entrada

1.2.1 Entradas: `Tarefa`

Campo	Tipo	Obrigatório	Descrição e restrições
`id_tarefa`	`int`	Sim	Identificador da tarefa. Inteiro positivo, não repetido no conjunto.
`data_criacao`	`datetime`	Sim	Abertura da tarefa. Ciente de fuso (timezone-aware).
`prazo`	`datetime`	Sim	Data e hora limite para a conclusão. Ciente de fuso.
`prioridade`	`str`	Sim	`"baixa"`, `"media"` ou `"alta"`, sem distinção de maiúsculas.
`status`	`str`	Sim	`"concluida"`, `"pendente"` ou `"cancelada"`, exatamente assim, em minúsculas.
`data_inicio`	`datetime | None`	Não	Início da execução. Quando informada, ciente de fuso e não anterior a `data_criacao`.
`data_conclusao`	`datetime | None`	Condicional	Obrigatória quando `status` é `"concluida"`. Ciente de fuso e não anterior a `data_criacao`. Se informada em tarefa não concluída, é validada, mas não entra em nenhum cálculo.

1.2.2 Saídas: `ResultadoAnalise`

Campo	Tipo	Descrição
`total_tarefas`	`int`	Quantidade de tarefas recebidas, de qualquer status.
`total_concluidas`	`int`	Quantidade com status `"concluida"`.
`total_pendentes`	`int`	Quantidade com status `"pendente"`.
`tempo_medio_conclusao_horas`	`float`	Média, em horas, de `data_conclusao - data_criacao` das concluídas. Duas casas decimais. `0.0` quando não há concluída.
`tempo_medio_por_prioridade_horas`	`dict[str, float]`	A mesma média, separada por prioridade. Sempre com as chaves `"alta"`, `"media"` e `"baixa"`. `0.0` para prioridade sem concluída.
`taxa_atraso_percentual`	`float`	Percentual das concluídas com `data_conclusao` depois do `prazo`, de `0.0` a `100.0`. Duas casas decimais. `0.0` quando não há concluída.

Tarefas canceladas entram em `total_tarefas`, mas não em `total_concluidas` nem em `total_pendentes`.

1.3 Regras de negócio

ID	Regra
RN-01	`total_tarefas` conta todas as tarefas; `total_concluidas` conta as de status `"concluida"`; `total_pendentes`, as de status `"pendente"`.

ID	Regra
RN-02	O tempo de conclusão de uma tarefa é `data_conclusao - data_criacao`, expresso em horas.
RN-03	Uma tarefa é atrasada quando `data_conclusao` é **estritamente** posterior a `prazo`. Conclusão exatamente no prazo não é atraso.
RN-04	Todas as datas precisam ser cientes de fuso. Data sem fuso é entrada inválida.
RN-05	Toda métrica de tempo e de percentual é arredondada para duas casas decimais **apenas na saída**, nunca no cálculo intermediário.
RN-06	O tempo médio é calculado no geral e por prioridade. A taxa de atraso é calculada no geral.
RN-07	`tempo_medio_por_prioridade_horas` traz sempre as três prioridades, com `0.0` quando não há concluída naquela prioridade.
RN-08	Nenhuma divisão é executada sem verificar antes se o denominador é zero.
RN-09	Lista vazia, ou lista sem nenhuma tarefa concluída, devolve `0.0` em todas as médias e na taxa de atraso, sem levantar exceção.
RN-10	Qualquer violação do contrato de entrada interrompe a análise e levanta `TaskValidationError`, com mensagem que identifique o `id_tarefa` e o campo responsável. Toda a entrada é validada antes de qualquer cálculo.
RN-11	A função é pura: não altera a lista recebida, não lê nem escreve arquivos, não acessa rede e não lê o relógio do sistema.

1.4 Violações que levantam `TaskValidationError`

- `id_tarefa` que não seja inteiro positivo, ou repetido no conjunto;
- `prioridade` fora de `"baixa"`, `"media"` e `"alta"`;
- `status` fora de `"concluida"`, `"pendente"` e `"cancelada"`;
- qualquer data sem fuso;
- tarefa `"concluida"` sem `data_conclusao`;
- `data_conclusao` anterior a `data_criacao`;
- `data_inicio` anterior a `data_criacao`.

2. Cenários de aceite e Test Harness

Os cenários são a tradução verificável do contrato. Cada um vira ao menos um teste em `tests/test_harness.py`.

2.1 Cenário 1: sucesso

Dado	seis tarefas válidas: quatro concluídas (duas de prioridade alta, uma média e uma baixa), uma pendente e uma cancelada. As concluídas levaram 2 h e 4 h (alta), 6 h (média) e 10 h (baixa), e duas delas terminaram depois do prazo
Quando	`analyze_tasks` for executada
Então	`total_tarefas` = 6, `total_concluidas` = 4, `total_pendentes` = 1, `tempo_medio_conclusao_horas` = 5.5, por prioridade `alta` = 3.0, `media` = 6.0 e `baixa` = 10.0, e `taxa_atraso_percentual` = 50.0

2.2 Cenário 2: exceção por datas inconsistentes

Dado	uma tarefa concluída cuja `data_conclusao` é anterior à `data_criacao`
Quando	`analyze_tasks` for executada
Então	deve levantar `TaskValidationError`, sem calcular métrica, com mensagem que cite o `id_tarefa` e o campo `data_conclusao`

2.3 Casos de borda

ID	Dado	Então
B-01	lista vazia	totais `0`, médias `0.0`, taxa `0.0`, as três prioridades com `0.0`, sem exceção
B-02	só tarefas pendentes	`total_tarefas` e `total_pendentes` iguais ao tamanho da lista, `total_concluidas` = 0, médias e taxa `0.0`, sem exceção

2.4 Cenários complementares

ID	Dado	Então
C-01	duas concluídas, uma exatamente no prazo e outra um minuto depois	`taxa_atraso_percentual` = 50.0
C-02	prioridade fora da enumeração	`TaskValidationError` citando o campo `prioridade`
C-03	prioridade escrita em maiúsculas, como `"ALTA"`	aceita e contada como `"alta"`
C-04	data sem fuso	`TaskValidationError`
C-05	tarefa concluída sem `data_conclusao`	`TaskValidationError` citando `data_conclusao`
C-06	`id_tarefa` repetido, zero ou negativo	`TaskValidationError` citando `id_tarefa`
C-07	status fora da enumeração	`TaskValidationError` citando `status`
C-08	média com dízima, como 1 h, 2 h e 2 h	valor arredondado em duas casas: 1.67
C-09	uma lista válida e uma cópia profunda dela	depois da execução, a lista original é igual à cópia
C-10	`data_inicio` anterior a `data_criacao`	`TaskValidationError` citando `data_inicio`

2.5 Critério de aceitação do harness

```
pytest -v
```

O comando deve terminar com **todos os testes aprovados e nenhum ignorado**. Enquanto isso não acontecer, o código gerado não é aceito, mesmo que pareça correto na leitura.

---

3. Governança de contexto

As regras para o assistente de inteligência artificial estão em `CONTEXT_RULES.md`, na raiz do repositório, e são fornecidas junto com esta especificação em toda solicitação.

4. Repositório e versionamento

4.1 Árvore

```
sdd-task-analyzer/
├── README.md                    # Visão geral, execução e status
├── CONTEXT_RULES.md             # Regras de governança da IA
├── .gitignore                   # Arquivos ignorados pelo Git
├── .dockerignore                # Arquivos fora da imagem
├── .github/
│   └── workflows/
│       └── ci.yml               # Pipeline de CI (MUD-13)
├── Dockerfile                   # Empacotamento (MUD-14)
├── docker-compose.yml           # Orquestração do serviço tests (MUD-14)
├── requirements.txt             # Dependências autorizadas (apenas pytest)
├── pyproject.toml               # Configuração do pytest (MUD-11)
├── specs/
│   ├── task_analyzer_spec.md    # Esta especificação, fonte da verdade
│   ├── system_design_document.pdf  # A mesma, para leitura (MUD-15)
│   └── requirements.yaml        # Requisitos para leitura por máquina (MUD-15)
├── tests/
│   └── test_harness.py          # Test Harness com pytest
└── src/
    └── task_analyzer.py         # Código gerado via IA e homologado
```

A especificação nasce primeiro e fica imutável durante a geração. O teste nasce em seguida e define o critério objetivo. O código é o último, e é o único dos três que pode ser descartado e regenerado a partir dos outros dois.

4.2 Homologação humana

Nenhum código gerado é aceito sem, nesta ordem:

1. `pytest -v` com todos os testes aprovados;
2. conferência campo a campo contra as seções 1.2, 1.3 e 1.4;
3. conferência contra cada item de `CONTEXT_RULES.md`;
4. revisão de legibilidade, coesão e tratamento de erro;
5. aprovação registrada no Pull Request, e só então o merge.

4.3 Versionamento

Aspecto	Definição
Branches	`main` estável, `develop` de integração (MUD-12) e uma `feature/<nome>` por entrega: `feature/sdd-specification`, `feature/test-harness`, `feature/task-analyzer-impl`, `feature/ci-pipeline`, `feature/docker` e `feature/specs-fase3`
Commits	pequenos e descritivos, no padrão Conventional Commits
Pull Requests	obrigatórios para entrar na `develop` e na `main`, com descrição, checklist de homologação e build verde na CI
Tags	uma por entrega avaliativa: `v1.0.0` ao final da Fase 2 e `v2.0.0` ao final da Fase 3

4.4 Integração contínua

O arquivo `.github/workflows/ci.yml` define o pipeline:

Etapa	Definição
Gatilhos	push ou Pull Request nas branches `main` e `develop`
Ambiente	`ubuntu-latest` com Python 3.11
Preparação	`pip install -r requirements.txt`
Execução	`pytest tests/test_harness.py -v`
Container	job `docker`, executado depois do anterior: `docker compose run --rm --build tests`

Build vermelho bloqueia o merge. Um Pull Request só entra com os dois jobs verdes.

4.5 Containerização

Arquivo	Papel
`Dockerfile`	constrói a imagem a partir de `python:3.11-slim`, instala só o `requirements.txt`, copia `src/` e `tests/` e define o pytest como comando principal (`CMD`). Roda com usuário sem privilégio de root.
`docker-compose.yml`	define o serviço `tests`, com sistema de arquivos só de leitura. A suíte roda com um único comando: `docker compose run --rm tests`.